F24-213-D-BinaryBeats
CampusEye: Distributed Anomaly Detection for Campus Safety

Project Team

Hamid Ishaq	21I-0476
Muhammad Abdullah	21I-2976
Muhammad Hasaam	21I-0698

Session 2021-2025

Supervised by

Mr. Muhammad Aadil Ur Rehman

Co-Supervised by

Dr. Muhammad Arshad Islam

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2024

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The aim of this project proposal is to give an outline framework for the real-time anomaly detection for improvement of security in universities from the CCTV footage. This document defines the goals and the limitations of the system, technical methodology, the time frame, and the expected results. It will also help to map the project's development by creating a relationship between the stakeholders and the developers of the system while outlining the detailed nature of the system, its capabilities, and how it shall be implemented.

1.1 Problem Statement

University and college campuses on the other hand pose a lot of challenges in providing security and safety to students because of the large and constantly changing population. In this case, the large number of the students, faculty, and visitors would mean that there are activities that are conducted in the various buildings and even the active learning spaces that needs to be monitored. The conventional closed circuit television surveillance systems are common in operations, but suffer from the acting of monitoring by security officers; hence, responded based on observed events only, resulting in late response to incidents and effective intervention. The management of camera feeds is a challenge that security teams struggle with and as such finding a way of monitoring every violation or anomalous event live is nearly impossible.

Ineffectiveness can be cited in matters that have to do with smoking in prohibited areas, violence or vandalism, frequent acts of policy infractions among others may not be a subject of attention until harm is caused. This type of response hampers the security of campuses and also erodes the capacity for the university to implement policies. Also, time spent watching hours of the recorded event to flag any suspicious activity is quite tiring, making it more of a challenge to the security staff.

The shortcomings of current systems are additionally compounded by the fact that increasing the scale of a physical infrastructure or hiring more people to enhance watchfulness can be expensive. Today's universities require an upgraded, more proactive and less costly approach to security threats management in real-time. Thereby, it is impossible to avoid the problem of uncoordinated deficiencies in security coverage within the territory of the campus, and students, staff, and property will remain vulnerable.

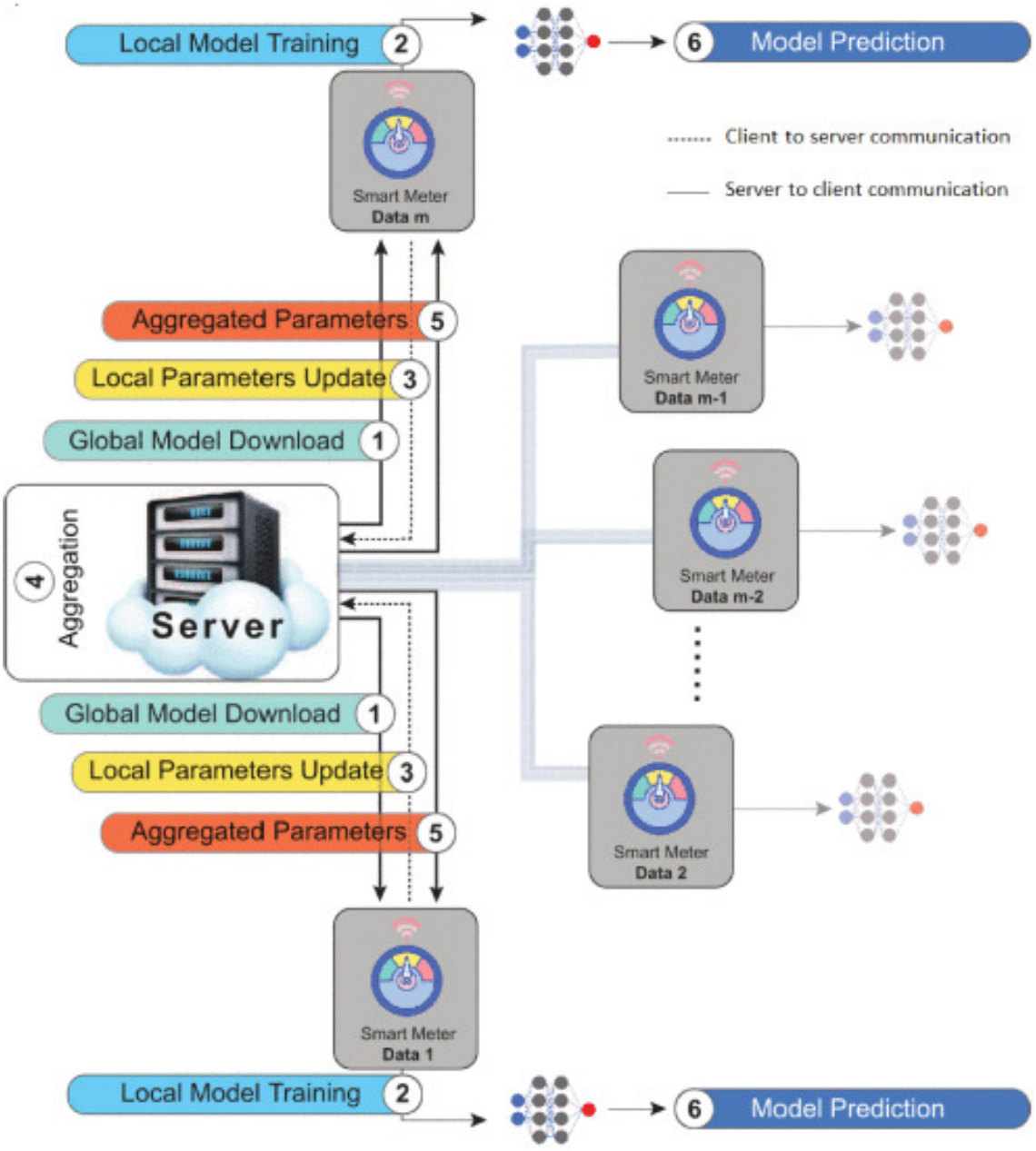
1.2 Scope

The notion of interest for the scope of this project is to create real-time anomaly detection for university campuses with an emphasis on security threats and policy breaking. The system will be installed on the top of the existing CCTV gadgets and camera will detect smoking, violence, vandalism and other unusual events using techniques of machine learning. Hence, the present system will harness distributed computing, whereby the computational burden will be spread across the other PCs in the university during their idle time, thus making it cheaper and scalable since the university will not be required to invest in other PCs.

Some of the important functions of the system comprise of video analysis in real time, giving out alarm to the security personnel, and making of video clips for occurrences detected. It will be possibile to categorize the kind of anomalies and send out alerts with info for further steps. An end-user audience that will be able to utilize security operations dashboards will include security personnel; Live Feeds, Security Alerts, and Incident Timeline will be available for viewing and analysis.

The system will emphasize the security area through the practices of automating processes of surveillance and bringing efficiency for faster reaction to situations. It will also be versatile so that, in the future, the students could expand on the implementation with regard to the number of CCTV cameras or rates of data transmission. Further, the solution shall incorporate with the current IT systems without stressing the current resources as well as cut on COST through optimum utilization of available resources. This project will only emphasize on identification of policies that have been violated and security threats that are applicable to university settings for instance; instances of vandalism, taking a puff in prohibited zones, and occurrences of fights.

Figure 1.1: Federated Learning[1, 2]



1.3 Modules

1.3.1 Dashboard & Monitoring Module

A user interface that security personnel using this software can use to monitor live video feeds, view detected anomalies, and manage system settings.

1. Displays live video feeds and detected anomalies.
2. Provides options to view incident clips, analyze data, and generate reports.
3. Allows configuration of system parameters and alert thresholds.

1.3.2 Video Input & Preprocessing Module

For handling the collection of video streams from the university's existing CCTV cameras, converting them into a format suitable for real-time analysis.

1. Captures and processes real-time video streams.
2. Preprocesses video data (resizing, normalization).
3. Handles multiple camera inputs and synchronizes video feeds.

1.3.3 Distributed Computing Module

This module distributes the computational load of training the model and detecting anomalies across idle PCs within the university network for cost-effective video processing.

1. Allocates tasks to idle machines for parallel video processing.
2. Ensures load balancing across distributed nodes.
3. Monitors and optimizes computational resource usage for efficiency.

1.3.4 Anomaly Detection Module

Fundamental Module for detecting anomalies such as smoking, vandalism, or violence using machine learning models.

1. Applies fine-tuned pre-trained models to classify anomalies in video feeds.

2. Identifies specific types of anomalies based on trained categories (smoking, vandalism, violence.).

3. Supports real-time analysis of streaming data.

1.3.5 Alert and Notification Module

Sends alerts to security personnel or other relevant authorities when an anomaly is detected.

1. Generates instant notifications upon anomaly detection.

2. Sends alerts via email or a dedicated dashboard.

3. Provides a timestamp and link to the detected video clip for quick review.

1.4 User Classes and Characteristics

Example: User Classes and Characteristics

Table 1.1: User Classes and description

User class	Description
1. Security Personnel	The system will primarily serve security staff who monitor the live camera feeds and react to anomalies. While these users possess basic technical competencies they should demand a user-friendly system that enables them to navigate live feeds and activate alerts. Ensuring safety on campus means they should act quickly on detected threats such as violence or vandalism recognised by the system. They must have methods for stopping and observing the reported issues immediately.
2. IT Administrators	The system's backend functions will be directed by IT personnel. They take charge of the nodes that process video data while maintaining the functioning of the anomaly detection algorithms. They show expertise in handling systems networks and secure protocols. They arrange the system set-up and check that updates reach the models as they conduct reliability and security inspections on the distributed network. User permission and role management falls under their duties.

3. University Administrators	Administrators at the university will utilize the system for an informed perspective on campus safety. Their participation in immediate feed tracking is limited; instead, they pay attention to reports, analysis, and overall system efficiency. The users will count on the system to aid in grasping the frequency and kinds of cases that arise on campus. Dashboards will help them merge data and produce reports on incidents to inform decisions about safety on campus and resource management. They aim to study lasting tendencies and the operation of the system as well as the consequence of the anomaly detection system on preserving order.
4. Maintenance Technicians	To guarantee the proper operation of CCTV hardware and communication networks technicians oversee maintenance. Access to a portion of system data including error logs and alerts about camera problems will be necessary for them although they will not directly work with the anomaly detection system. Some users might need specific permissions to check system conditions and repair crashed cameras. The focus of their knowledge is on supporting and resolving the physical framework and they make sure the monitoring devices function efficiently for the system to run at its peak.
5. Students and Faculty	In cases linked to their own involvement or to boost transparency in enforcement the system will be accessible to students and faculty only in limited ways. When students or faculty document an incident they could examine video related to that instance. Their access will probably be very limited when focusing on particular use cases such as reporting disputes or confirming claims. Engagement with the system's functions may happen through messages or submissions related to campus safety programs.

Chapter 2

Project Requirements

2.1 Event Response Table

Table 2.1: Event Response Table

Event	Source	Response	Destination
Video stream starts	CCTV Cameras	Live video is delivered to the master node.	Master Node
Task distribution initiated	Master Node	The master node divides the video stream into smaller chunks and sends them to nodes.	Distributed
Node receives video chunk	Distributed Nodes	Node starts handling the provided video chunk for identifying anomalies with the existing model.	Local Node Processing
Anomaly detected in video chunk	Distributed Nodes	Upon classifying the anomaly, a notification containing metadata is communicated to the master node.	Master Node
No anomaly detected in video chunk	Distributed Nodes	An update shows that the master node that no anomaly falls within acceptable parameters.	Master Node
Anomaly detection results aggregated	Master Node	All nodes contribute their results to the master node that prepares anomaly information for alerting.	Master Node Processing

Notification sent for detected anomaly	Master Node	Alerts the UI dashboard about the anomaly type and accompanying details (time and location).	UI Dashboard (Security Personnel)
Video clip extraction initiated	Master Node	The master node collects the relevant video clip from the anomaly detection and stores it for analysis.	Storage System
Video clip ready for review	Master Node	Link to the video clip is notified to security personnel for additional review and response.	Security Personnel
User views anomaly details and video clip	UI Dashboard (Security)	Users responsible for security review the details and clip and respond with corrective steps.	Security Personnel
Node failure detected	Distributed Nodes / Master	The system rearranges the task for the failed node with another active node to guarantee ongoing processing.	Master Node, Other Nodes
New camera feed added to system	CCTV Cameras	The master node configures the new camera feed for inclusion in the tasks assigned process.	Master Node, Distributed Nodes
Daily/weekly report generated	Master Node	The master node prepares an overview of detected anomalies and system performance overview.	University Administrators
Idle node detected	Master Node	Reallocation tasks to idle nodes enhances performance and lessens processing time.	Master Node, Distributed Nodes

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature. It is just one possible way to arrange them. Other organizational options include arranging functional requirements by use case, process flow, mode of operation, user class, stimulus, and response depend on what kind of technique has been used to understand functional requirements. Hierarchical combinations of these elements are also possible, such as use cases within user classes.

2.2.1 Dashboard Monitoring Module

Following are the requirements for Dashboard Monitoring Module:

1. **FR1:** The system shall display real-time video streams from all connected CCTV cameras on this dashboard.
2. **FR2:** The system shall display detected anomalies on the dashboard, with time and locations.
3. **FR3:** The system shall allow users to access and view stored detected anomalies as video clips.
4. **FR4:** The system shall allow users to generate reports that contain detected anomalies and node performance.
5. **FR5:** The system shall allow users to configure system settings such as alert and detection parameters.

2.2.2 Video Input Preprocessing Module

Following are the requirements for module Video Input Preprocessing Module:

1. **FR1:** The system shall get live video streams from all connected CCTV cameras.
2. **FR2:** The system shall preprocess the video feeds by resizing and normalizing the input data.
3. **FR3:** The system shall handle and synchronize video feeds from multiple cameras for real-time processing.

2.2.3 Distributed Computing Module

Following are the requirements for Distributed Computing Module:

1. **FR1:** The system shall distribute the computational load(Video Stream) for anomaly detection across available idle machines.
2. **FR2:** The system shall monitor the performance of distributed nodes to ensure load balancing.
3. **FR3:** The system shall reallocate tasks in the event of a node failure to maintain uninterrupted processing.

4. **FR4:** The system shall leverage the Federated Learning to train the model and aggregate results to the Master Node.

2.2.4 Anomaly Detection Module

Following are the requirements for Anomaly Detection Module:

1. **FR1:** The system shall apply a machine learning model to classify anomalies in the real-time video stream.
2. **FR2:** The system shall detect and classify anomalies based on predefined trained labels such as smoking, vandalism, or violence.
3. **FR3:** The system shall ensure the detection of anomalies in real time within a few seconds of their occurrence.

2.2.5 Alert and Notification Module

Following are the requirements for Alert and Notification Module:

1. **FR1:** The system shall generate instant high-level alerts when an anomaly is detected.
2. **FR2:** The system shall notify the relevant authorities via email or the monitoring dashboard, including a video clip and details like occurrence time and location.
3. **FR3:** The system shall allow users to configure alert preferences, such as frequency of report and method of notification.

2.3 Non-Functional Requirements

Criteria outside basic functionality are established by non-functional requirements to optimize system quality and performance. Its reliable operation depends on these essential requirements for both effectiveness and user satisfaction. Here are some NFRs for the real-time anomaly detection system using distributed computing:

2.3.1 Performance

- The system ought to evaluate incoming video data promptly with little delay.

- Available resources must be evenly divided across the machines to guarantee efficient use and minimize time waits.

2.3.2 Scalability

- This system shall accommodate new video streams and increase computational requirements by increasing the number of nodes while maintaining performance.

2.3.3 Reliability

- The system ought to provide reliable operation and have low downtime. Upon failure of a node, this system will distribute tasks to different idle nodes to maintain continuous processing.
- To ensure that anomaly detection remains constant, the system needs to ensure maximum uptime at all times.

2.3.4 Usability

- Access to the interface should be simple and intuitive for those handling security roles that need only basic training.
- Deliveries of alerts should inform appropriate details and offer simple ease of interaction.

2.3.5 Maintainability

- The design aims to make the system manageable for maintenance providing the means for updating detection models and security patches without prolonged interruptions.

2.3.6 Compatibility

- This system needs to align easily with the current CCTV video setup and should not need any adjustments to its current configuration.
- It accommodates diverse hardware configurations present in various university PCs dedicated for distributed processing.

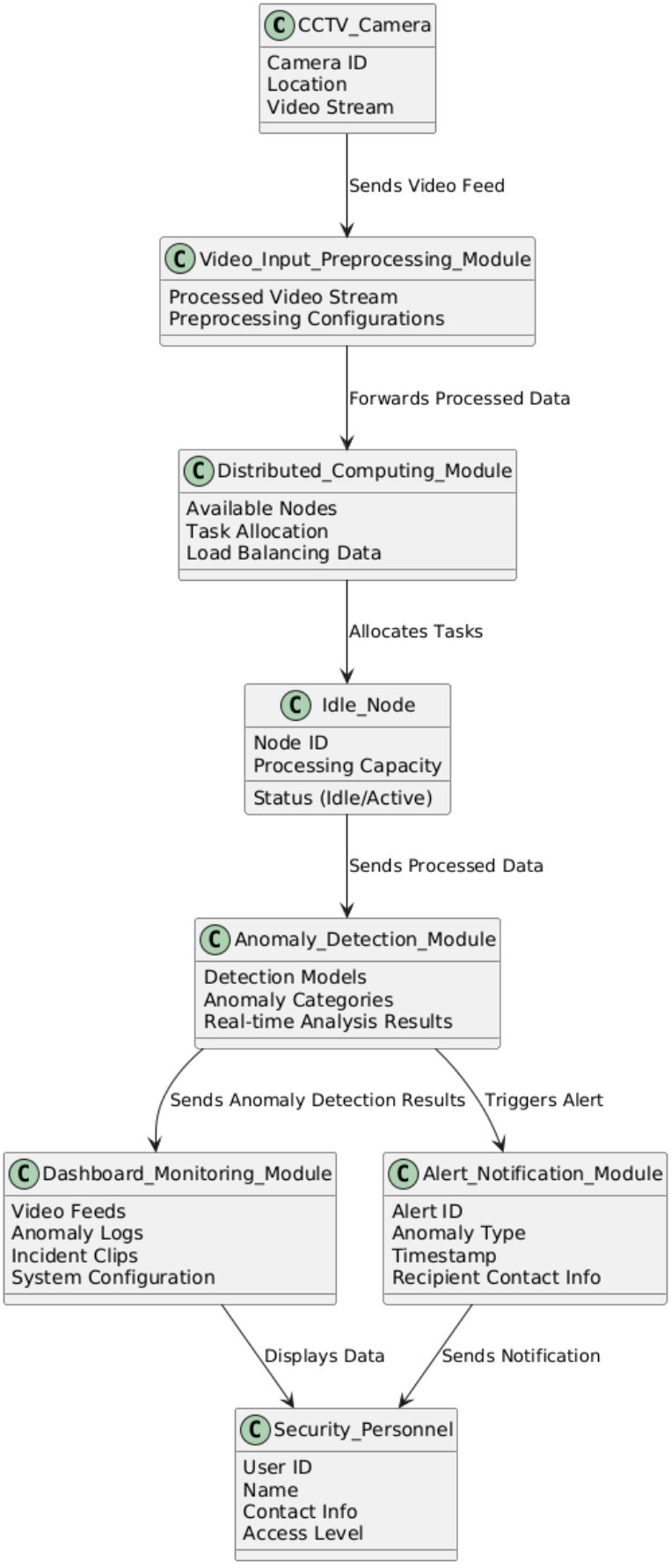
2.3.7 Fault Tolerance

- The system needs to react to hardware or software breakdowns such that one failure does not affect overall efficiency.
- When a node fails, the system needs to re-assign duties to usable nodes for continual anomaly identification.

2.4 Domain Model

- **CCTV Camera:** Sends the video stream to the Video Input & Preprocessing Module.
- **Video Input and Preprocessing Module:** Processes and passes video data to the Distributed Computing Module.
- **Distributed Computing Module:** Divides work into chunks to idle Nodes for computing.
- **Idle Nodes:** Send processed data to the module that detects anomalies.
- **Anomaly Detection Module:** Recognizes anomalies and conveys results to both the Alert and Notification Modules and the Dashboard and Monitoring Modules.
- **Dashboard and Monitoring Module:** Displays live video and various alerts to the security authorities.
- **Alert and Notification Module:** Sends notifications along with relevant details to the relevant authorities.

Figure 2.1: Domain Model



Chapter 3

System Overview

3.1 Architectural Design

We are using the Master-Slave architecture for our real-time anomaly detection system because it provides an efficient way to distribute computational tasks across multiple nodes, enabling scalable and concurrent processing of video streams. In this architecture, the master node coordinates the system by dividing the video data into smaller chunks and assigning them to worker nodes, which independently perform anomaly detection using machine learning models. The results are then aggregated and managed by the master node. This approach ensures efficient use of resources, load balancing, and fault tolerance, allowing the system to handle multiple video streams in real-time while maintaining centralized control and coordination.

The modular program structure of your system, as depicted in the diagram, involves several key components working together:

1. **Desktop Application:** This provides the end-user (for instance the administrators) with the platform to engage with the system, watch the CCTV footage and also receive notifications or even alarms.

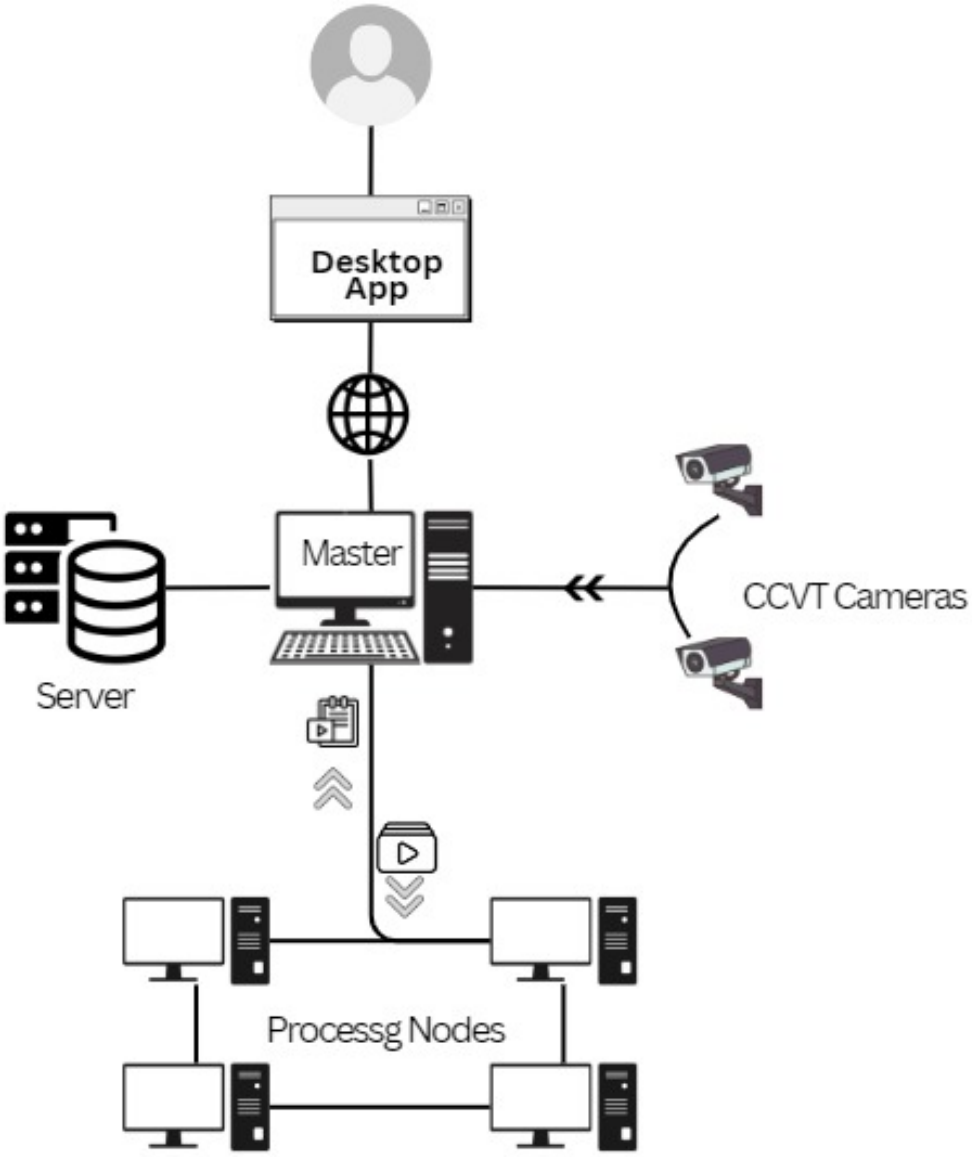
2. **Master Node:** This is the controller that coordinates the tasks between the server, CCTV cameras and the processing nodes.

3. **Server:** Records videos, maintains logs of system activities, and the data that goes to analysis. The slave node works independently to send requests to the master node for data either to receive or transmit.

4. **Processing Nodes:** These implement distributed computing of video data to check for above mentioned anomalies in real-time with computational benefit.

5. **CCTV Cameras:** Recording real-time streams of videos with motion and conveying the captured streams to the master node.

Figure 3.1: Architectural Design

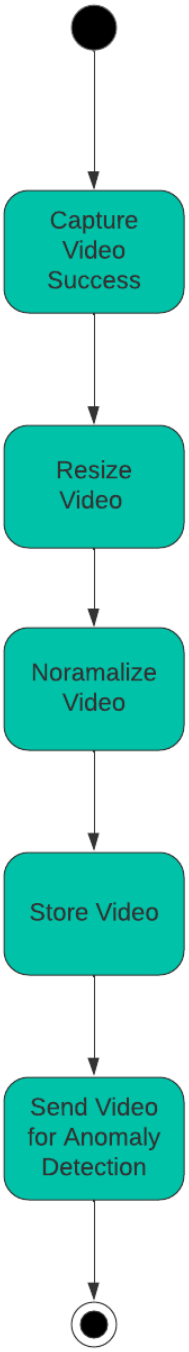


3.2 Design Models

3.2.1 Activity Diagrams

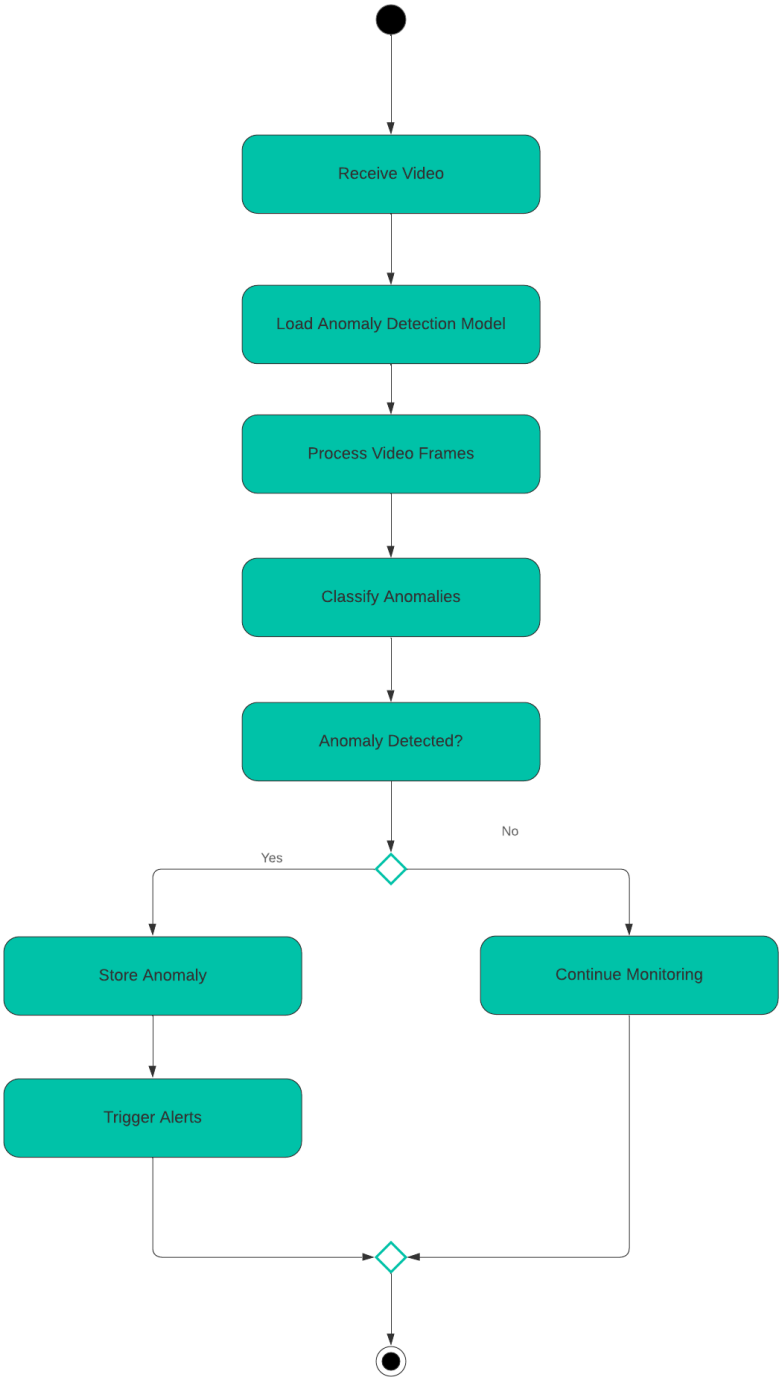
1. **Video Input and Preprocessing Activity Diagram**

Figure 3.2: Video Input and Preprocessing Activity Diagram



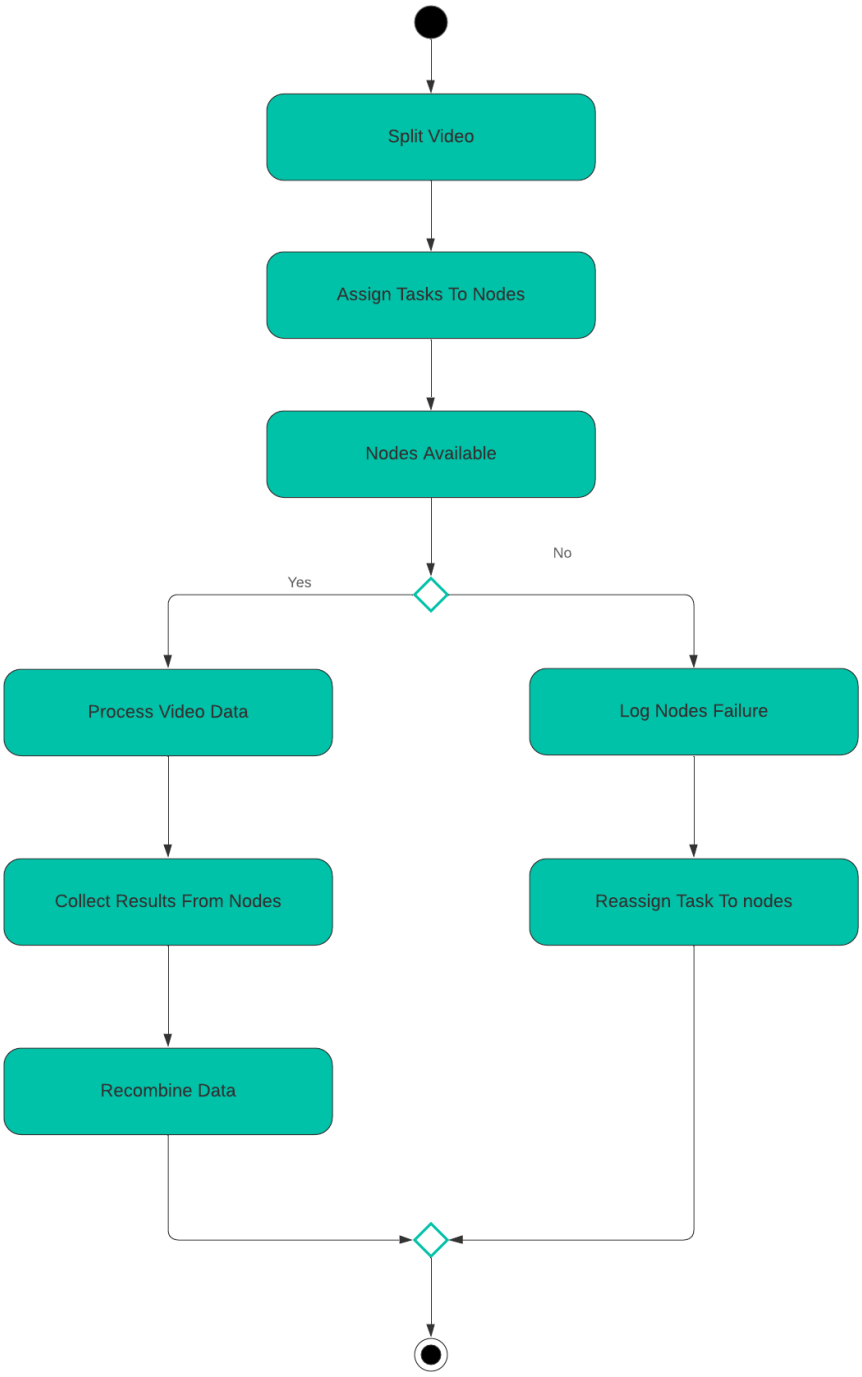
2. **Anomaly Detection Activity Diagram**

Figure 3.3: Anomaly Detection Activity Diagram



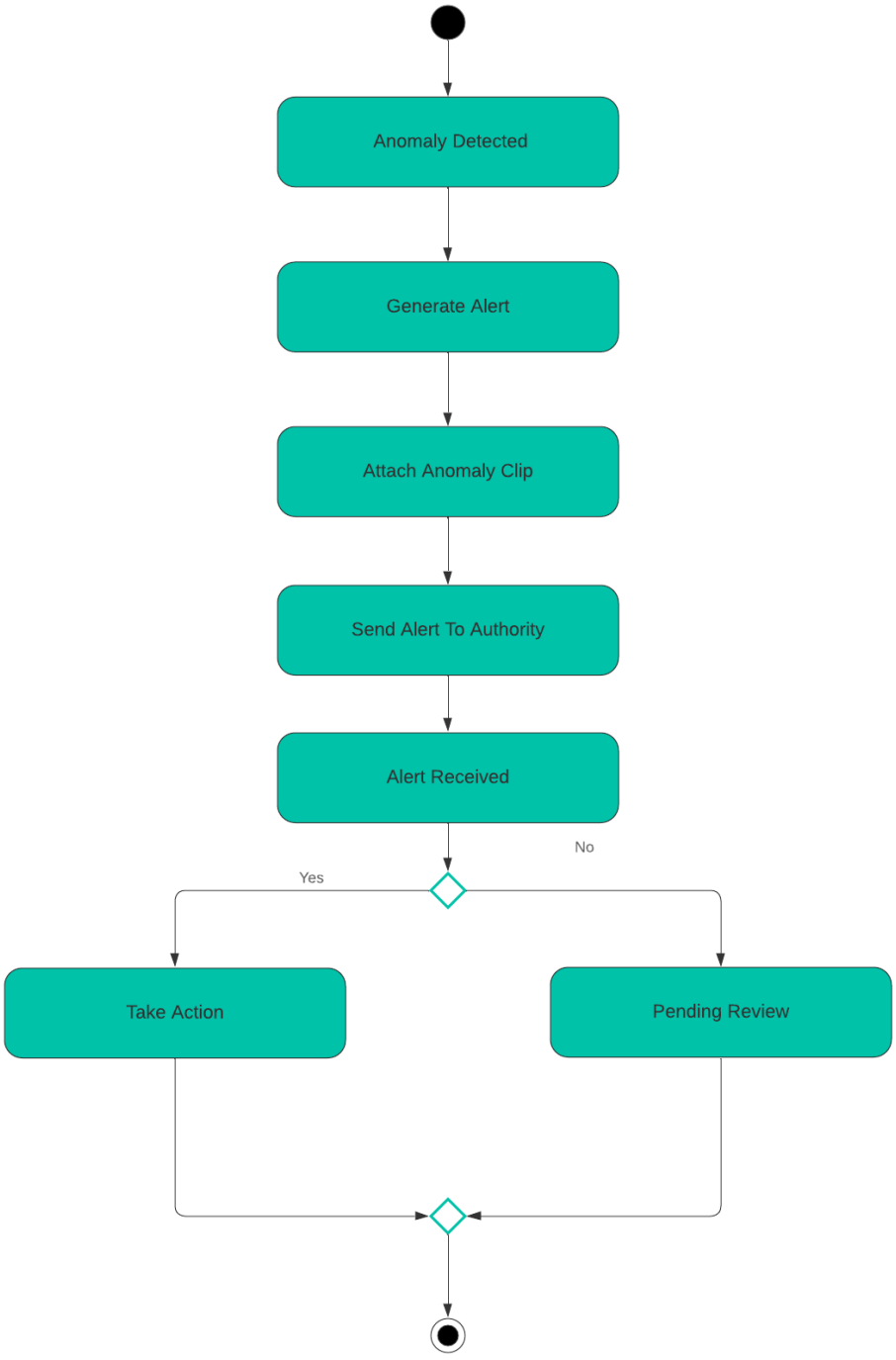
3. **Distributed Computing Task Allocation Activity Diagram**

Figure 3.4: Distributed Computing Task Allocation Activity Diagram



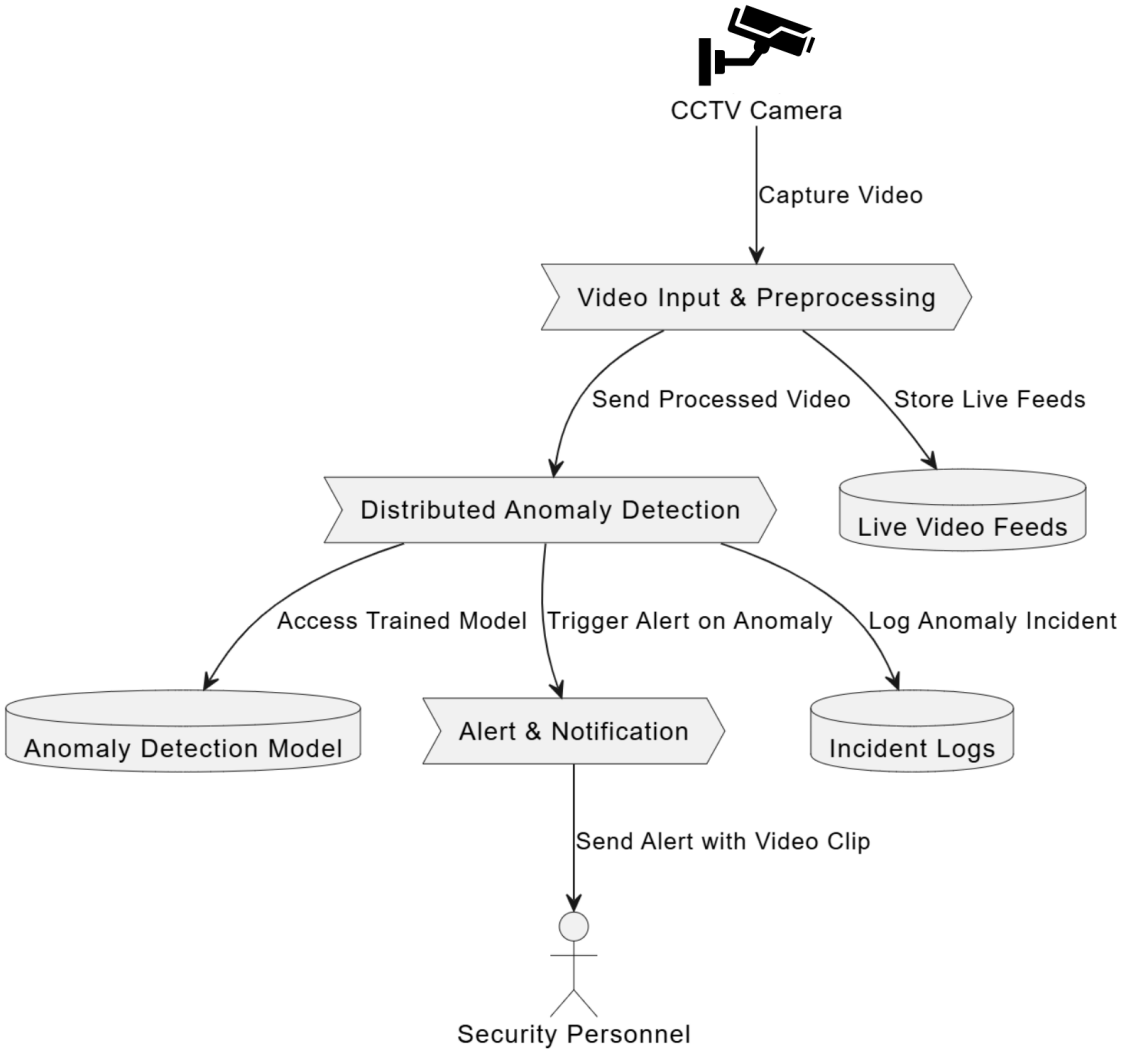
4. **Alert and Notification Activity Diagram**

Figure 3.5: Alert and Notification Activity Diagram



3.2.2 Data Flow Diagram

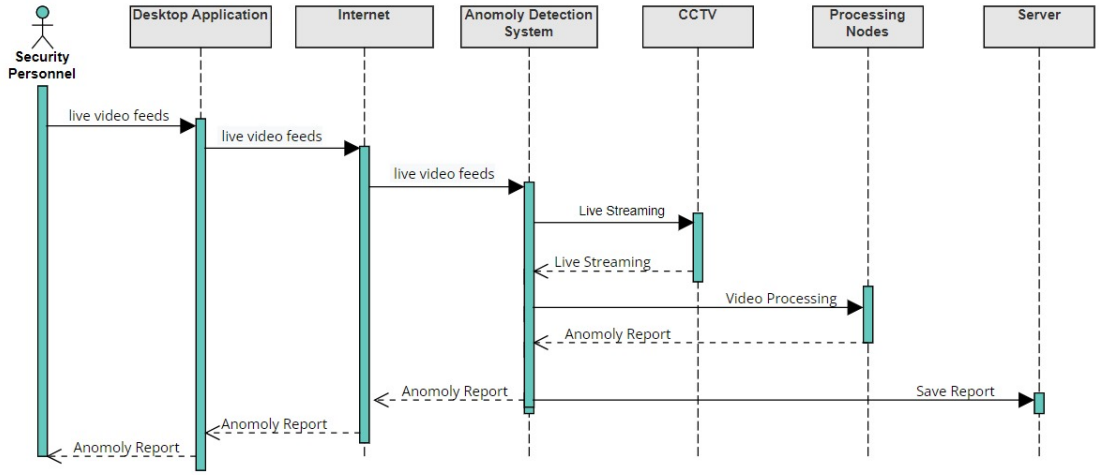
Figure 3.6: Data Flow Diagram



3.2.3 System Sequence Diagram

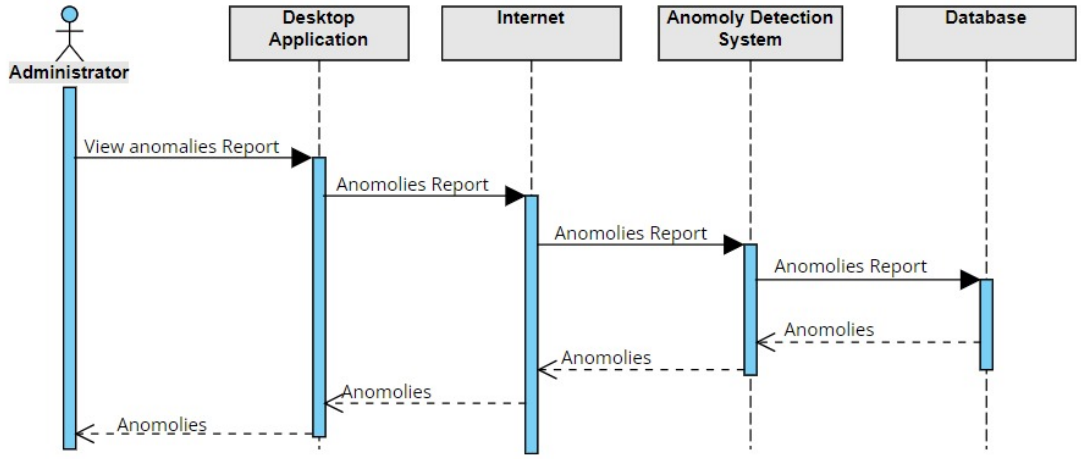
1. **Live Video Feeds**

Figure 3.7: Live Video Feeds



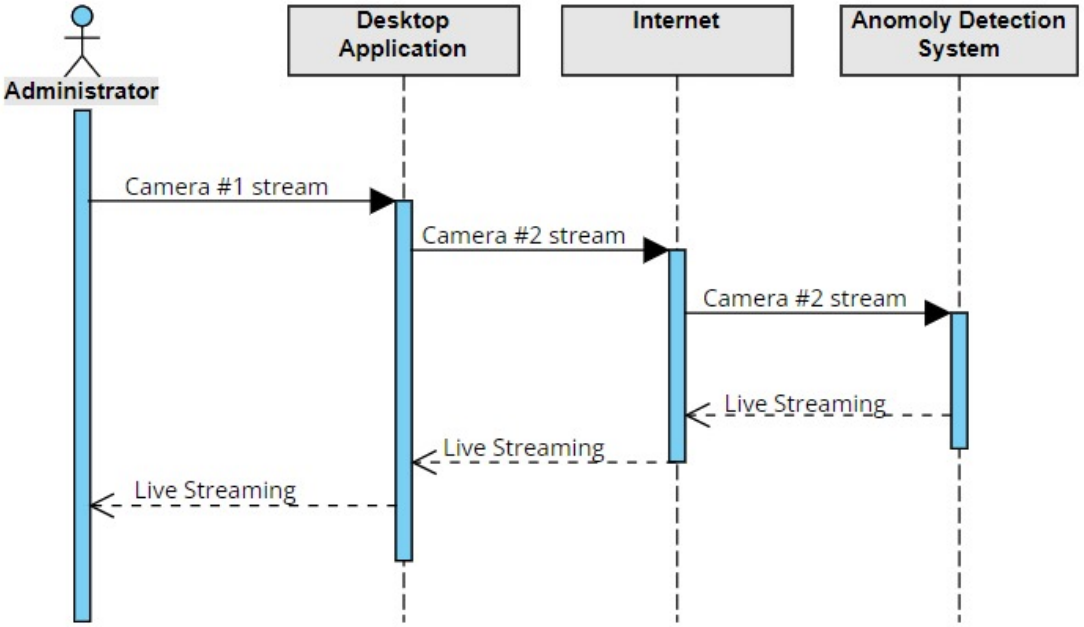
2. **View All anomalies Report**

Figure 3.8: View All anomalies Report



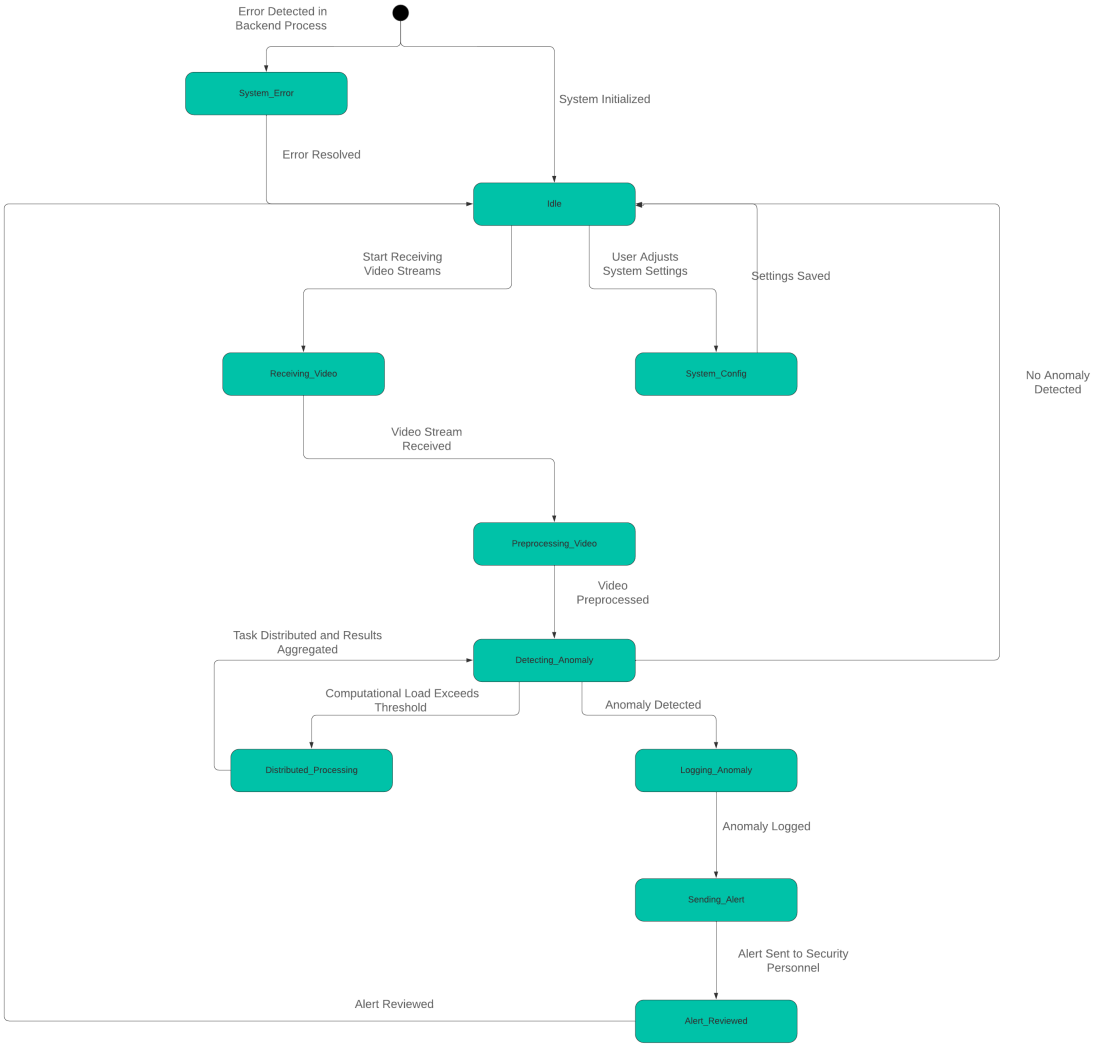
3. **View Live Streaming**

Figure 3.9: View Live Streaming



3.2.4 State Transition Diagram

Figure 3.10: State Transition Diagram



3.3 Data Design

In this anomaly detection system, the information domain mainly concerns CCTV video streams, anomaly logs, and interaction of the system users. Data extracted from the information domain is converted and warehoused in a suitable form to allow for fast analysis. Since the system is built on MongoDB which is a NoSQL database, the data is stored in a relatively free-form, schema-less structure where collections contain documents instead of rows and tables to facilitate easy horizontal expansion and to work well with large volumes of data that are not strictly columnar and often incomplete.

3.3.1 Data Storage and Organization

1. **Video Streams**

   - **Raw Input:** Raw data consist of captured video streams from CCTV cameras thereafter resized and normalized for further processing.
   - **Data Processing:** For this, the frames taken from the video stream are input to the trained machine learning models for anomaly detection.
   - **Storage:** Video frames are not saved as they are in MongoDB, also the processed video data are also not stored in MongoDB. However, information in the form of time, location and source of the video stream are stored in the MongoDB database. If anomalies are present, then the system gets the specified clips of the video and stores them either locally or in a cloud solution, with only references residing in the MongoDB.

2. **Anomalies and Logs**

   - **Structure:** Any anomalies found are noted by the machine learning models and recorded together with other data such as:
     (a) Anomaly Type (such as smoking or vandalism).
     (b) Timestamp
     (c) Location by camera ID, by building
     (d) Video Clip Reference (the anomaly clip)
     (e) Status (for example “Reviewed,” “Pending Review”)
   - **Organization:** This data is enlisted as documents in a MongoDB collection which is for instance anomaly logs. An individual document is an anomaly and contains all the meta-information associated with that anomaly.

3. **Users and Permissions**

   - **User Roles:** It is also used by several user classes including the security, administrative and the monitoring staffs
   - **Structure:** Users are documents in a users collection and example of this collection includes:
     (a) User ID
     (b) Name
     (c) Responsibility (for example Security staff, Administrative staff)
     (d) Contact Information
     (e) Access rights (under which access level and with what sensitivity can change system functionalities).

4. **Distributed Computing Tasks**

   - **Task Allocation:** Computational workloads like task processing and video processing jobs running in nodes of distributed system.
   - **Storage:** Notification regarding assignment of tasks, handling of nodes and results of processing are stored in the MongoDB to track the status of tasks, failures in nodes and utilization of resources. This information can be stored in a task logs collection, with fields such as:
     (a) Task ID
     (b) Node Assigned
     (c) Project Work Progress (e.g., "Active," "Done").
     (d) Processing Time
     (e) Node Performance Metrics

5. **Alerts and Notifications**

   - **Structure:** Anomalies themselves cause alerting and the alerts are saved in the alerts collection. Each document includes:
     (a) Alert ID
     (b) The Anomaly Reference to the particular anomaly that instigated the alert to be formed is also included.
     (c) Timestamp
     (d) Security Personnel Contact
     (e) Notification Type (for instance "Sent" or "Read").

3.3.2 MongoDB Data Structure

In MongoDB, documents stored in collections are represented in JSON-like format. Below is an example of how the data might be organized:

Example Documents in MongoDB Collections

1. **anomaly_logs Collection**

   {
   "_id": "anomaly_id_001",
   "type": "Smoking",
   "timestamp": "2024-09-30T10:45:00Z",
   "location": {

"camera_id": "cam_101",

"building": "Main Hall"

},

"clip_reference": "/videos/anomalies/smoking_001.mp4",

"status": "Pending Review"

}

2. **users Collection**

{

"_id": "user_id_001",

"name": "John Doe",

"role": "Security Personnel",

"contact": {

"email": "i210123@nu.edu.pk",

"phone": "03123456789"

},

"permissions": ["View Feeds", "Review Anomalies", "Receive Alerts"]

}

3. **alerts Collection**

{

"_id": "alert_id_001",

"anomaly_ref": "anomaly_id_001",

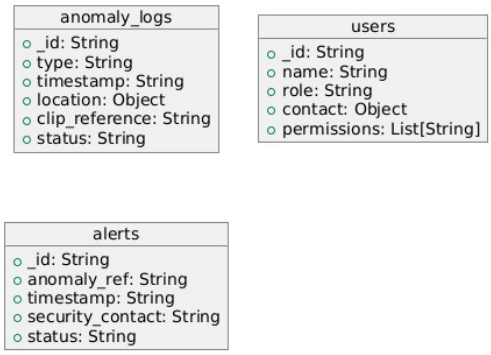
"timestamp": "2024-09-30T10:46:00Z",

"security_contact": "i210123@nu.edu.pk",

"status": "Sent"

}

Figure 3.11: Data Representation diagram for JSON schema



3.3.3 Data Flow

1. **Capture and Preprocess Video:** It records video streams and its raw formats undergo transformations. The results of metadata extraction and the actual anomaly clips only are placed in MongoDB, but the videos might be located elsewhere.

2. **Real-Time Anomaly Detection:** Video frames are preprocessed and filtered against precalculated models, making finds are stored in anomaly logs and alerts.

3. **User and Task Management:** Users' actions are recorded, distributed computing operations are performed, the data concerning the task distribution is saved in MongoDB.

Bibliography

[1] Kim Y. Jeong CS Kim, YK. Ride:real-time massive image processing platform on distributed environment. *Image Video Proc*, 2018.

[2] A Kolyshkin and S Nazarovs. Stability of slowly diverging flows in shallow water. *Mathematical Modeling and Analysis*, 2007.